

Dag Versioning in Airflow: Version Proliferation and Open Questions

Ephraim Anierobi

Senior Software Engineer @ astronomer.io
Committer & PMC member of Apache Airflow



Airflow Remembers your dag history

One Dag. Version 42. what happened?

Agenda

01

Why the number
keeps climbing

02

What disabling
bundle versioning
actually does

03

What code really
runs when you
clear a task

04

The parts nobody
has solved yet.

“We deployed once. Why did version 8 grow to 42?”

```
DagVersion.version_number
```

=

the order in which Airflow preserved Dag versions

≠

a Git revision, a release number, or a compatibility promise

It answers “**which recorded version came first?**” — not “what changed?” and not “is this safe?”

A high number is not automatically a defect. An unexplained climbing number is.

Three identities, three questions



Source artifact

What files were delivered?

Git SHA · latest local files



Dag Version

Which serialized state was preserved?

16, 17, 19



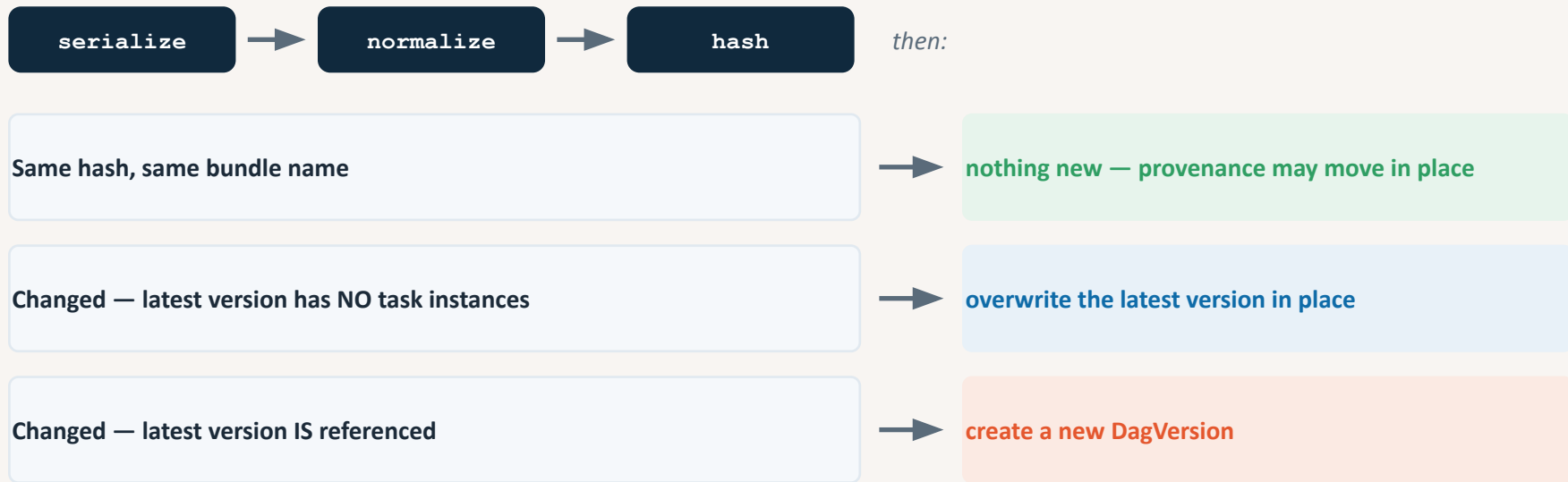
Bundle Version

Which code version should the worker load?

SHA or None

Every confusion in this talk is a collapse of these three into one number.

What creates a version – the whole rule



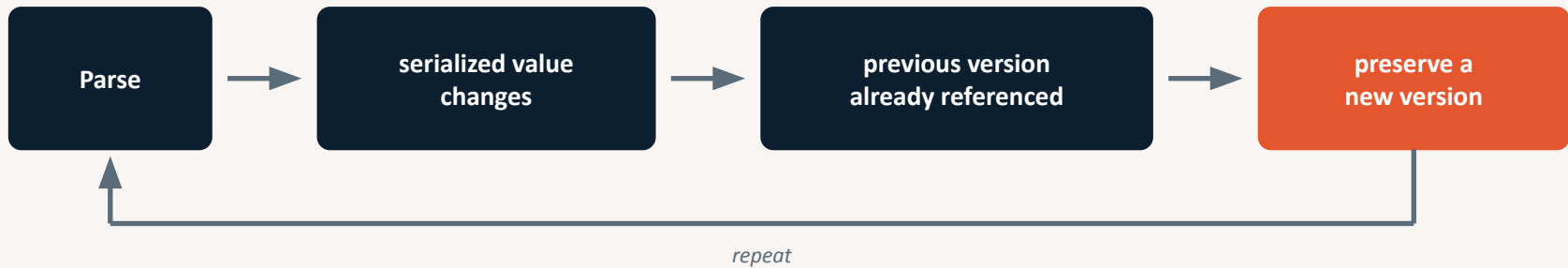
Task-instance references turn mutable “current” state into history.

What does not create a version

Change	Why not
Git commit only; serialized Dag unchanged	Dag version provenance refreshes in place
New <code>bundle_version</code> / <code>version_data</code> , same content	lives on DagVersion, not in the hash
File path move	excluded from the hash
Dictionary key reordering	hash canonicalizes ordering
Code change inside a task function body	function bodies are not serialized

Not every deploy creates a version — and not every version means a deploy.

Proliferation is a loop



Both conditions must hold:

the hash keeps changing

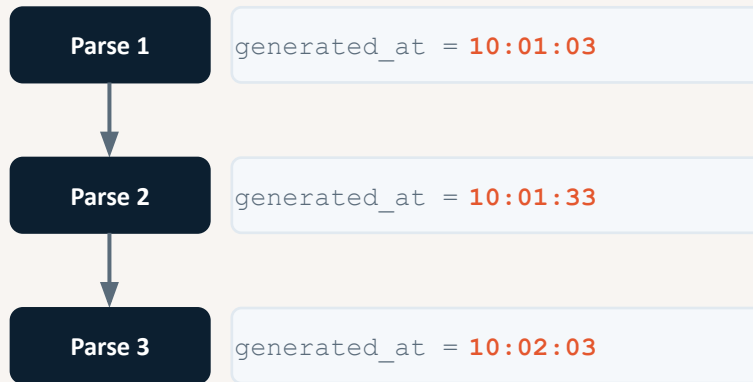
AND

the latest version is already referenced

Root cause: unstable serialized output — not the counter.

The dynamic Dag trap

```
with DAG(  
    dag_id="payments",  
    params={  
        "generated_at":  
            datetime.now().isoformat(),  
    },  
):  
    ...
```



→ three hashes, three version rows, zero deployments

A changing parse-time value in serialized constructor arguments changes the hash.

Not always your fault



Author-side

- Parse-time `now()`, random, UUIDs
- Environment & API reads at parse
- Duplicate `dag_ids` across files



Airflow-side (fixed bugs)

- Object repr with memory addresses
- Path-derived callable names
- Representation mismatches between entry points

If the changing value is an Airflow-internal representation — report it, don't rewrite the Dag around it.

The static checker

Core Configuration Parameter

```
[dag_processor]dag_version_inflation_check_level
```



What It Detects

Identifies common sources of dynamic DAG inflation:

- Standard **time**-based generator calls
- Non-deterministic **random** values
- Dynamic **UUID** generation at parse time



Blind Spots

Misses complex or indirect code execution:

- Environment reads at parse time
- Custom utility helpers and abstractions
- Airflow-internal serializer representation bugs

Diagnose before you change bundle policy

- 1 **Confirm** the version count and timing
- 2 **Compare** consecutive serialized states
- 3 **Locate** the changing field
- 4 **Reproduce** two parses from the same source
- 5 **Check** whether the previous version had task instances

Do not start by deleting history or disabling bundle versioning.

disable_bundle_versioning=True does not disable Dag versioning

It changes

- whether a dag run records a bundle version
- whether worker asks for historical source

It does not change

- serialized Dag hashing
- the referenced-vs-unreferenced decision
- DagVersion creation

Think “latest source policy” — not “stop version rows.”

Clears, reruns, backfills: two independent axes

	Historical	Latest
Dag Version	keep the task's dag_version_id	explicitly move to latest
Bundle Version	use the run's bundle version	current bundle when no pin

Ask both questions, in order — they are answered independently.

Historical DagVersion + no bundle version = only partially reproducible.

Where the bundle version comes from: delivery in one slide

Helm git-sync

`dags.gitSync → LocalDagBundle (default)`

NO SOURCE PIN

Kubernetes-level file sync into a volume. Airflow sees an unversioned local folder; every clear, rerun, and backfill executes the latest synced code.

GitDagBundle

`dagBundleConfigList → providers.git`

PINNED: COMMIT SHA

Airflow owns fetch and versioning. Runs record the SHA; workers rematerialize that exact commit — clears land in the reproducible column.

git-sync is Dag delivery. GitDagBundle is Dag delivery *with versioning semantics Airflow can see.*

What is already fixed

- ✓ Deterministic ordering before hashing; fileloc / bundle_name excluded **3.0.6**
- ✓ Provenance refreshed in place for unchanged content **3.3.0**
- ✓ atomic version / serialized-Dag / Dag-code writes **3.1.4**
- ✓ Serializer determinism fixes — memory addresses, callable names, deadline UUIDs **3.1.7, 3.2.x, 3.3.0**
- ✓ Static parse-time checks; duplicate-dag_id warnings **3.2.0/main**

The direction: normalization plus evidence — not turning history off.

Open question

What Airflow can already do: **preserve history safely.**

What it cannot yet do:

- 1 explain what changed — and why a version was created
- 2 say what kind of change it was
- 3 give non-Git sources a real version identity
- 4 agree on where bundle version state should live

Can Airflow explain “why”?

What changed?

Observed-state diff of two stored versions — categorized, deterministic, honest that it reports current stored state.

IN REVIEW · PR #69864

[1 · Versions](#)[2 · Safe diff](#)[3 · CODE details](#)[4 · Honest boundary](#)

Apache Airflow · Design preview

Dags / customer_daily_summary

customer_daily_summary

Daily customer revenue and quality checks

Active

[Overview](#)[Grid](#)[Graph](#)[Runs](#)[Versions](#)

Dag versions

2 selected · compare 41 → 42

[Compare versions](#)

Select	Version	Created	Bundle version	Runs
☒	42	Aug 15, 09:42	b2178ae	8
☒	41	Aug 14, 17:06	9f4b71d	23
☐	40	Aug 14, 08:13	5ab80c2	19
☐	39	Aug 13, 15:31	3221dc9	12

[1 · Versions](#)[2 · Safe diff](#)[3 · CODE details](#)[4 · Honest boundary](#)

Apache Airflow · Design preview

Dags / customer_daily_summary

customer_daily_summary

Active

Daily customer revenue and quality checks

[Overview](#)[Grid](#)[Graph](#)[Runs](#)[Versions](#)

Version 41 → Version 42

4 changes · serialized schema 3 → 3

Observed state

Show values

CODE access

Path	Operation	Category	Impact
/dag/tasks/*/retries	changed	task	execution
/dag/tasks/*	added	task	execution
/dag/tags/*	added	metadata	metadata
/provenance/bundle_version	changed	provenance	provenance

Source comparison

Unavailable for historical comparison

[1 · Versions](#)[2 · Safe diff](#)[3 · CODE details](#)[4 · Honest boundary](#)

Apache Airflow · Design preview

Dags / customer_daily_summary

customer_daily_summary

Active

Daily customer revenue and quality checks

[Overview](#)[Grid](#)[Graph](#)[Runs](#)[Versions](#)

Version 41 → Version 42

4 changes · serialized schema 3 → 3

Observed state

Values shown

CODE access

Path	Operation	Category	Impact	Before	After
/dag/tasks/extract_orders/retries	changed	task	execution	1	3
/dag/tasks/validate_schema	added	task	execution	—	task added
/dag/tags/finance	added	metadata	metadata	—	finance
/provenance/bundle_version	changed	provenance	provenance	9f4b71d	b2178ae

Source comparison

Unavailable for historical comparison

[1 · Versions](#)[2 · Safe diff](#)[3 · CODE details](#)[4 · Honest boundary](#)

Apache Airflow · Design preview

Dags / customer_daily_summary

customer_daily_summary

Active

Daily customer revenue and quality checks

[Overview](#)[Grid](#)[Graph](#)[Runs](#)[Versions](#)

Version 41 → Version 42

4 changes · serialized schema 3 → 3

Observed state

Values shown

CODE access

Path	Operation	Category	Impact	Before	After
/dag/tasks/extract_orders/retries	changed	task	execution	1	3
/dag/tasks/validate_schema	added	task	execution	—	task added
/dag/tags/finance	added	metadata	metadata	—	finance
/provenance/bundle_version	changed	provenance	provenance	9f4b71d	b2178ae

Source comparison

Unavailable for historical comparison

Compares currently stored state; not creation-time evidence.

Compact Evidence

Why was this row created?

Compact evidence — reason codes, digests, referenced-or-not — captured in the same transaction as the new row.

DRAFT PROPOSAL

Airflow can overwrite the latest version in place. So a diff run later may not show what was there at the time

Semantic Dag versions?

Integer version: which state came first?

Semantic annotation: what kind of change?

major

minor

patch

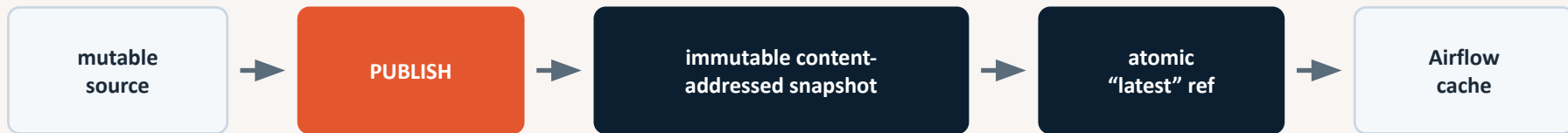
unknown

← anything the classifier cannot prove is
unknown — never silently safe

Airflow cannot infer business compatibility from arbitrary Python, custom operators, or callbacks. Classification should explain existing history before it is ever allowed to influence write policy, clears, or backfills.

Annotate history first. Do not change creation policy.

Version identity beyond Git



```
version = sha256 of the canonical manifest — self-certifying, fails closed
```

Rejected: hashing a mutable live tree on every refresh — races, implicit releases, cost & repo size. Publication must be a deliberate boundary.

Changing files is not a deployment until publication moves the reference.

The working proof: airflow-manifest-bundle

It already exists — outside the Airflow tree

- v0.4.0, released on github
- zero core changes; stock Airflow 3.0+
- local and S3, GCS adapters; the artifact store itself can be an S3/gcs prefix
- the version string alone is the whole contract



[github.com/ephraimbuddy/
airflow-manifest-bundle](https://github.com/ephraimbuddy/airflow-manifest-bundle)

Feasibility is settled. Where it belongs is not.

Where should bundle version state live?

	2025 dev-list proposal	S3 bundle(open #70207)	out-of-tree 0.4.0
Manifest home	Provider DB table	Core version_data	External artifact store
DagVersion impact	none	new rows per deploy	none
Status	WITHDRAWN	OPEN · UNMERGED	SHIPPED · NOT CORE

2025 dev-list consensus: provider state belongs in the provider's storage. Unresolved — watch the dev list.

What you can do today



Dag authors

- Keep parse-time inputs deterministic
- Variability belongs in task execution
- Treat version growth as a signal, not noise



Platform teams

- Choose latest-code or pinned-source explicitly
- Test clear / rerun / backfill before incidents
- Retain pinned artifacts while runs can request them
- Monitor version growth per Dag

“Follow latest” should be a decision — not a default nobody noticed.

A version number tells us which state.

Still open: what changed · why · from which artifact · what remains unknown



Questions?

Presentation Slides

Scan to access this presentation



[Slide Link](#)

Airflow Manifest Bundle

Scan to explore the source repository



<http://github.com/ephraimbuddy/airflow-manifest-bundle>



Connect

GitHub

github.com/ephraimbuddy

LinkedIn

www.linkedin.com/in/ephraimanierobi

The 2026 Apache Airflow® Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)

